Internship Project Report: V-Guard Smart Device Integration and Embedded Security

Sachin Chandela · 6 November 2026

1. Introduction and Project Overview

This report covers two internship projects at V-Guard, one that brings V-Guard devices into the Apple Home app and one that builds a web app for the inverter, plus a security review of the embedded platforms behind them.

Item	Detail
Intern	Sachin Chandela
Mentor	Prasad M V
Reporting manager	Narender Singh Negi

The work falls into three parts:

1. **V-Guard devices on the Apple Home app.** A custom Home Assistant integration lets a user sign in with a V-Guard account, fetch their devices, and expose them to Apple Home through the HomeKit Bridge.
2. **Web app for the inverter.** A browser-based configuration interface with a login page and tabs for ISP, remote server, certificate and Wi-Fi settings.
3. **Cybersecurity study.** A review of common software vulnerabilities and a comparison of the security features of the ESP32-C3, ESP32-C5 and STM32H5 chips against the current V-Guard firmware builds.

2. Background Technologies

The Apple Home project rests on four technologies: the Apple Home app, the Matter standard, Home Assistant and its HomeKit Bridge. V-Guard devices do not implement Matter, so a bridge on the Home Assistant side is the route into Apple Home.

2.1 Apple Home app

The Home app is Apple's interface for controlling smart home accessories from an iPhone, iPad or Mac. It works with HomeKit accessories and with Matter-certified accessories, and it can be used with Siri.

2.2 Matter

Matter is an open, universal smart-home standard developed by the Connectivity Standards Alliance. It runs over IP networks, including Wi-Fi and Thread, so that devices from different brands can work together (ST Blog: Understanding Matter).

Without Matter, a manufacturer writes separate code for Apple HomeKit, Amazon Alexa, Google Home and Samsung SmartThings. With Matter, one firmware that follows the specification works with all of them. The V-Guard devices in this project were not built with Matter firmware, which is why a different route was needed.

2.3 Home Assistant

Home Assistant is open-source software that acts as a central control hub for a smart home. It represents each device capability as an *entity* (for example a fan or a switch) and lets users add new device types through *integrations*.

2.4 HomeKit Bridge

HomeKit Bridge is a built-in Home Assistant integration that makes Home Assistant entities available in Apple Home and to Siri, including devices that do not support HomeKit natively (Home Assistant documentation). Anything that appears as an entity in Home Assistant can therefore be controlled from the Apple Home app.

The documentation lists these practical limits:

- One bridge supports at most 150 unique accessories, so larger setups need more than one bridge.
- The phone must be on the same local network as Home Assistant while pairing.
- The network must allow mDNS (UDP 5353) and the bridge port (TCP 21063 by default).
- Cameras, locks, TV media players and activity-based remotes need *accessory mode*, with one entity per bridge.

2.5 Custom integrations and config flows

A custom integration is Python code that teaches Home Assistant about a new brand or device family. Its *config flow* is the dialog a user goes through to add the integration from the interface: it collects the required details, checks that they work, and then creates a config entry (Home Assistant developer docs). In this project the config flow is where the V-Guard sign-in happens.

Technology	Role in this project	Key point
Apple Home app	End-user control on iPhone	Controls HomeKit and Matter accessories
Matter	Cross-ecosystem standard	Not implemented in current V-Guard firmware
Home Assistant	Hub that hosts the V-Guard integration	Open source, entity-based
HomeKit Bridge	Exposes entities to Apple Home	Up to 150 accessories per bridge

3. Project 1: V-Guard Devices on the Apple Home App

The project makes V-Guard devices appear in the Apple Home app by writing a custom Home Assistant integration and passing its devices on through the HomeKit Bridge. A prototype showed a V-Guard fan entity, called APHONIC, on both Home Assistant and an iPhone.

3.1 Problem and objective

V-Guard already has a mobile app that communicates with its products and devices. Apple Home users, however, manage their other accessories in the Home app. The objective is to let a user sign in once with a V-Guard account, pull in all of their devices, and control them from the Home app alongside everything else.

The obvious route would be Matter, but the current V-Guard firmware does not include the Matter stack. Adding it would mean new firmware on every device. The bridge approach needs no change on the device side.

3.2 Solution approach

The task is to build a custom integration for Home Assistant. The user signs in, the integration fetches the devices tied to that account, and each device becomes a Home Assistant entity. HomeKit Bridge then publishes those entities to Apple Home. Figure 1 shows the full path.

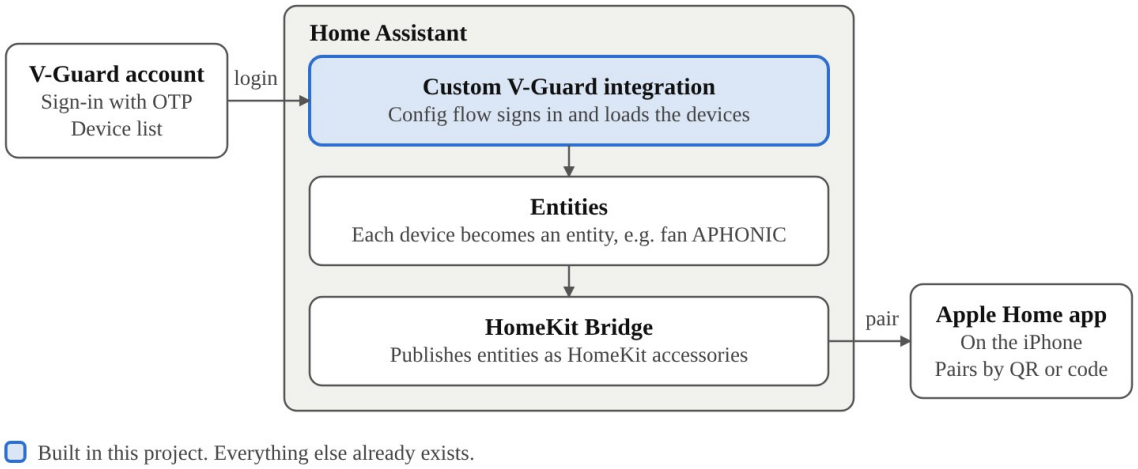


Figure 1. Apple Home integration architecture: only the custom integration had to be built.

The sign-in and the device list come from the V-Guard account; Home Assistant turns each device into an entity; HomeKit Bridge then hands those entities to Apple Home.

3.3 Implementation walkthrough

The steps below follow the prototype shown in the original presentation.

1. **Find the custom integration.** In Home Assistant, open Settings, then Devices & services, and search for the integration. The prototype appears as *Test_Devices*, version 0.1.0, labelled as a custom integration.
2. **Add an entry.** A new integration has no entries yet. Selecting *Add entry* starts the config flow.
3. **Sign in with a V-Guard account.** The flow shows a V-Guard branded sign-in page. The user chooses email or phone, requests a one-time password (OTP) by email, and enters it.
4. **Create the device entry.** After sign-in, an entry named *My Devices* is created and the account's devices are loaded.
5. **Check the entities.** The Entities page lists the fan APHONIC under the Test_Devices integration. Opening it shows a standard Home Assistant control with an on/off slider.
6. **Expose devices through HomeKit Bridge.** On the HomeKit Bridge page, a bridge entry is created and Home Assistant raises a pairing notification with a QR code and a setup code.
7. **Pair with the iPhone.** In the Apple Home app, the user adds the accessory by scanning the QR code or typing the code, and confirms the prompt for an uncertified accessory.
8. **Control from Apple Home.** The Home app now shows APHONIC in the default room, and tapping it opens the fan control.

3.4 Result and limitations

The prototype confirms the full chain from V-Guard sign-in to an Apple Home tile. Points to keep in mind:

- HomeKit Bridge only exposes entity types it supports, such as lights, switches, fans, locks and sensors (Home Assistant documentation).
- Each bridge is limited to 150 accessories.
- Users need their own Home Assistant instance, so this suits advanced users and pilot setups more than a mass-market rollout.

4. Project 2: Inverter Web App

The second project is a web interface for configuring the inverter’s connectivity. It has a login page and four tabs: ISP, Remote Server, Certificate and Wi-Fi. The original deck shows the interface only, so how it is hosted and what it talks to is not covered here.

4.1 Login

The app opens on a V-Guard branded login page with a username field, a password field and a Submit button. Every other page sits behind this login, which makes it the first line of defence against the *Broken Access Control* and *Authentication Failures* risks described in Section 5.

4.2 Configuration tabs

Tab	Fields	Purpose
ISP	APN, current SIM detail, username, password	Sets how the inverter's cellular connection reaches the internet
Remote Server	Remote server credentials	Uploads the credentials the inverter uses to connect to its remote server
Certificate	Root CA, Client CA, private key (file upload)	Loads the certificates the device uses to trust a server and to prove its own identity
Wi-Fi	Wi-Fi credentials	Uploads the credentials the inverter uses to join a Wi-Fi network

4.3 Key terms

- **APN (Access Point Name).** The gateway setting a SIM card uses to connect a device to a mobile network's internet service. The ISP tab asks for it together with the SIM details and the carrier credentials.
- **Root CA.** The certificate of the authority the device trusts. It lets the device confirm that the remote server's certificate is genuine.
- **Client certificate and private key.** Together they let the device prove its own identity to the server, which is called mutual TLS authentication. The private key must stay secret.

4.4 Design considerations

The Certificate tab handles the most sensitive data in the app, so it deserves the most care.

- Serve the app over HTTPS so credentials and keys are not sent in clear text.
- Never display a stored private key or password back in the page.
- Validate and size-limit uploaded files, and treat their content as untrusted input.
- Log configuration changes so a suspicious change can be traced.

These points map directly to the vulnerability categories listed next.

5. Cybersecurity: Common Software Vulnerabilities

The deck lists nine common software vulnerabilities, which match the OWASP Top 10:2025 categories except one: *A06 Insecure Design* is missing from the deck, so it is added below to complete the list of ten. OWASP publishes this ranking of the most critical application security risks.

OWASP 2025 category	Meaning	Where it could show up in these projects
A01 Broken Access Control	The application does not properly restrict what each user may access.	An inverter web page reachable without login, or one user seeing another user's devices.
A02 Security Misconfiguration	Insecure defaults or settings, such as showing a detailed error instead of a generic one.	Default passwords, verbose error pages, debug interfaces left enabled.
A03 Software Supply Chain Failures	Trusting third-party apps, libraries or plugins without checking them.	Libraries inside the firmware, the web app or the Home Assistant integration.
A04 Cryptographic Failures	Sensitive data is not encrypted, or weakly encrypted.	Wi-Fi, SIM and server passwords or private keys stored or sent in clear text.
A05 Injection	User input is treated as a command instead of plain text.	Configuration fields such as APN or username passed unchecked into commands or queries.
A06 Insecure Design	Missing security thinking at design time (added; not in the deck).	A configuration feature built without a threat model.
A07 Authentication Failures	Weak sign-in, with no extra verification step such as a second factor.	The inverter app uses only a username and password.
A08 Software or Data Integrity Failures	The application trusts updates or data without verifying them.	Over-the-air firmware updates that are not signature-checked.
A09 Security Logging & Alerting Failures	Suspicious activity is not recorded or acted on.	No log of failed logins or certificate changes.
A10 Mishandling of Exceptional Conditions	Poor error handling, such as overflows or failed memory allocation, and failing open.	A malformed upload crashing the device or leaving it unprotected.

Two of these connect straight to the hardware comparison in Section 6. A08 is the reason secure, signed firmware updates matter, and A04 and A07 are the reason certificate-based authentication matters.

6. Embedded Platform Security Comparison

The ESP32-C3, ESP32-C5 and STM32H5 chips all offer strong built-in security, but the current V-Guard ESP32 builds do not use several of those features. The deck compares the chips' capabilities with what the V-Guard C3 and C5 firmware actually enables.

6.1 Comparison table

Concept	ESP32-C3	ESP32-C5	STM32H5	V-Guard C3	V-Guard C5
Root of trust, secure boot	Yes	Yes	Yes	Yes	Yes
Secure firmware update	Yes	Yes	Yes	No	No
Anti-rollback	Yes	Yes	Yes	No	No
Early power-on protection	Yes	Yes	Yes	Yes	Yes
Modifiable attestation registers	No	No	No	No	No
Writable boot ROM	No	No	No	No	No
Failed boot-check handling	Yes	Yes	Yes	No	No
Native chain of trust	Yes	Yes	Yes	Partial	Partial
Secure recovery	Yes	Yes	Yes	Partial	Partial
Secret isolation	Limited	Yes	Yes	Yes	Yes
TPM-like key protection	eFuses	Key manager	Own	eFuses	Same as C5
TEE	Basic	Real	Real	No	No
OTA update	Yes	Yes	Yes	Yes	Yes
Certificate-based auth	Yes	Yes	Yes	No	No

The ESP32-C3, ESP32-C5 and STM32H5 columns show what each chip supports (the ESP32 columns also cover the Mini modules). The two V-Guard columns show what the current V-Guard firmware builds enable. For *Modifiable attestation registers* and *Writable boot ROM*, "No" is the secure answer, because these should not be changeable.

6.2 What each concept means

- **Root of trust and secure boot.** A trusted, unchangeable first stage checks the signature of the next stage before running it, so only approved firmware starts.
- **Chain of trust.** Each boot stage verifies the next, so trust extends from the chip to the application. *Partial* means some links are not verified.
- **Secure firmware updateability and OTA.** OTA delivers new firmware over the air; it is *secure* when images are signed (and optionally encrypted) and checked before install. The V-Guard builds support OTA but not the secure variant.
- **Anti-rollback.** A version counter stored in hardware stops an attacker from reinstalling an older, vulnerable firmware. On ESP32 the security version sits in eFuse and is compared at boot and during OTA (Espressif security overview).
- **Handling of failed boot checks.** What the device does when verification fails, for example refusing to run or entering a safe state.
- **Secure recovery.** A trusted way to restore a working firmware after a failed update or boot.
- **Early power-on protection.** Security settings take effect from the first moment of boot, before application code runs.
- **Secret isolation.** Keys and other secrets cannot be read by ordinary application code.
- **TPM-like key protection.** A dedicated place to keep keys. ESP32 chips have no separate TPM chip: the C3 uses eFuses, and the deck notes the C5 adds a key manager with crypto acceleration.

- **TEE.** A protected execution area, separate from the normal application, that holds keys and sensitive operations. Espressif's ESP-TEE is documented for the ESP32-C5 and provides secure storage, signed attestation tokens and secure updates (ESP-TEE guide). The deck rates the C3 as *Basic*, which corresponds to its permission-based memory protection.
- **Certificate-based authentication.** The device proves its identity with a certificate and private key, as in mutual TLS, instead of a shared password.

6.3 Gaps in the current V-Guard builds

The chips support these features, but the V-Guard builds do not use them:

1. **Secure firmware updates**, so an update may not be signature-checked (OWASP A08).
2. **Anti-rollback**, so an old vulnerable firmware could be reinstalled.
3. **Handling of failed boot checks**, so a failed check does not lead to a defined safe state.
4. **Certificate-based authentication**, which leaves password-style authentication as the main option (OWASP A04, A07).
5. **A TEE**, so secrets are not held in a protected area.

The chain of trust and secure recovery are only partly in place.

6.4 Reference point: STM32H5

The STM32H5 shows what a full implementation looks like. Its STiRoT immutable root of trust runs first after every reset, checks application authenticity with ECDSA-P256 and integrity with SHA256, supports encrypted updates, and uses a security counter for anti-rollback. It targets SESIP level 3 certification (ST application note AN6007).

6.5 Recommendations

Espressif recommends enabling Secure Boot on all production devices and using Flash Encryption in release mode with a unique key per device (Espressif security overview). Building on that, the next steps are:

1. Enable Secure Boot v2 and Flash Encryption in release mode on production units.
2. Sign OTA images, deliver them over HTTPS, and turn on the anti-rollback security version.
3. Define what happens when a boot check fails and how a device recovers.
4. Use certificate-based authentication, with the private key protected by hardware (the ESP32-C3 Digital Signature peripheral keeps the key out of software reach).
5. Evaluate ESP-TEE on the ESP32-C5 for key storage and attestation.

7. Conclusion and Future Work

The internship shows that V-Guard devices can reach the Apple Home app today without Matter firmware, by using a custom Home Assistant integration and the HomeKit Bridge. The security review shows that the hardware under those devices is stronger than the firmware currently uses it.

Suggested next steps:

1. Extend the prototype integration so it loads every device type on a V-Guard account, not only the fan, and add a re-authentication step for when a login expires.
2. Apply the design considerations in Section 4.4 to the inverter web app, especially to the credential and certificate uploads.
3. Close the firmware gaps in Section 6.3, starting with signed OTA updates and anti-rollback.
4. Consider Matter-capable firmware on newer chips such as the ESP32-C5 as a long-term replacement for the bridge.

8. References

All pages below were opened while preparing this report.

1. HomeKit Bridge integration, Home Assistant
2. Config flow, Home Assistant developer documentation
3. OWASP Top 10:2025, Introduction
4. Security Overview for ESP32-C3, Espressif ESP-IDF Programming Guide
5. ESP-TEE User Guide for ESP32-C5, Espressif ESP-IDF Programming Guide
6. AN6007: Getting started with STiRoT for STM32H5 MCUs, STMicroelectronics
7. Why does it MATTER? Understanding the universal protocol, ST Blog